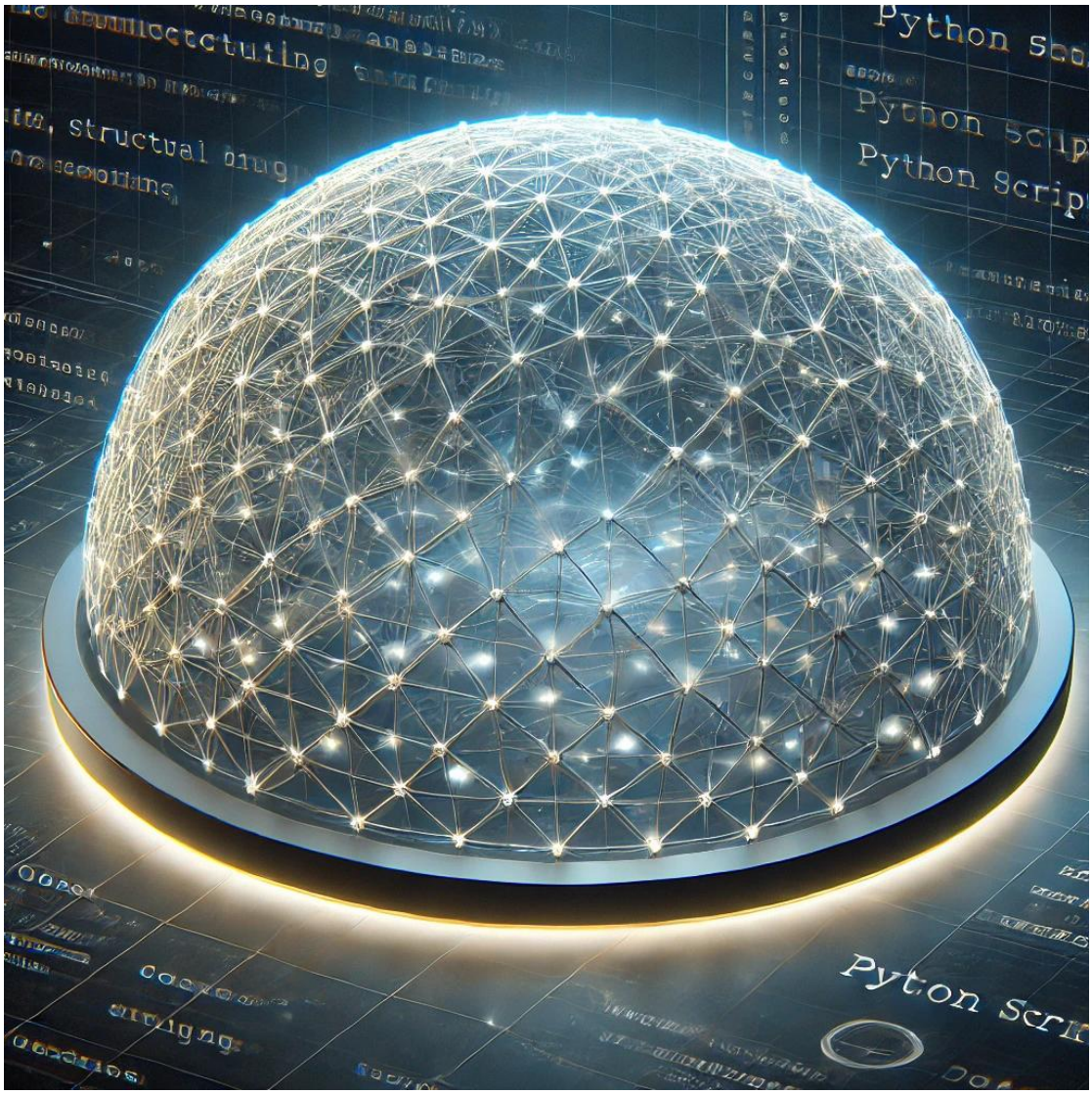


Tutorials

-CLUSTERS
-ABAQUS SCRIPTING

02/16/2025

Clusters

In this manual, we will explore the utilization of two of Purdue University's high-performance computing clusters: **Negishi** and **Bell**. These clusters are part of Purdue's Community Cluster Program, designed to support a wide range of computational research needs.

Negishi Cluster: Named in honor of Nobel laureate and Purdue professor Ei-ichi Negishi, this cluster is optimized for traditional, tightly-coupled science and engineering applications. Built in partnership with Dell and AMD, Negishi comprises Dell compute nodes, each equipped with two 64-core AMD Epyc "Milan" processors (totaling 128 cores per node) and 256 GB of memory. All nodes feature 100 Gbps HDR Infiniband interconnects, ensuring high-speed data transfer. The cluster also includes nodes with larger memory capacities and nodes equipped with AMD Instinct MI210 GPUs, catering to diverse computational requirements.

rcac.purdue.edu

Bell Cluster: Named after Clara Bell Sessions, a minority advocate and former Director of Continuing Education in Nursing at Purdue, the Bell cluster is similarly optimized for tightly-coupled scientific and engineering tasks. Constructed through a collaboration with Dell and AMD, Bell consists of Dell compute nodes, each housing two 64-core AMD Epyc "Rome" processors (128 cores per node) and 256 GB of memory.

rcac.purdue.edu

Both clusters utilize the Slurm workload manager for resource and job management, providing users with a robust and flexible environment for executing a variety of computational tasks. Access to these clusters is offered on a per-processor basis, allowing researchers to scale their computational resources according to their specific project needs.

In the following sections, we will delve into the specifics of accessing these clusters, managing computational resources, and optimizing your workflows to fully leverage the capabilities of Negishi and Bell. Even tough, the clusters may seem different they share the same principles when it comes to using them. All the demos and examples will be from Negishi.

ACCESSING THE CLUSTER

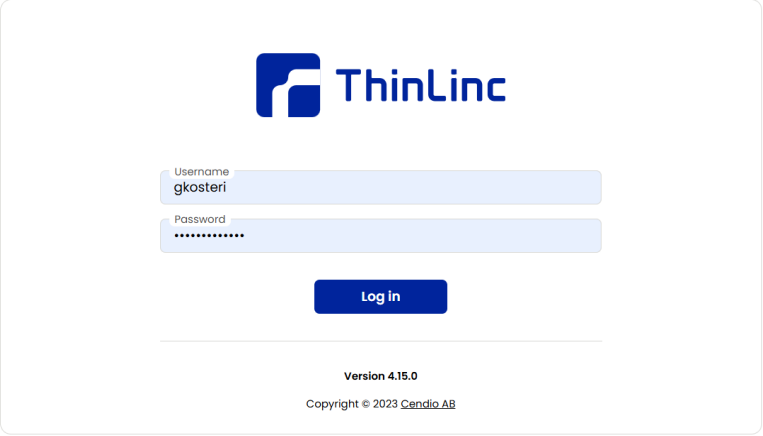
Accessing Purdue's high-performance computing clusters, such as Negishi and Bell, can be achieved through two primary methods: the **remote desktop option via the web portal** and **SSH using Visual Studio Code (VS Code)**. Both methods provide secure and efficient ways to connect to the clusters and manage your computational tasks.

Remote Desktop Option

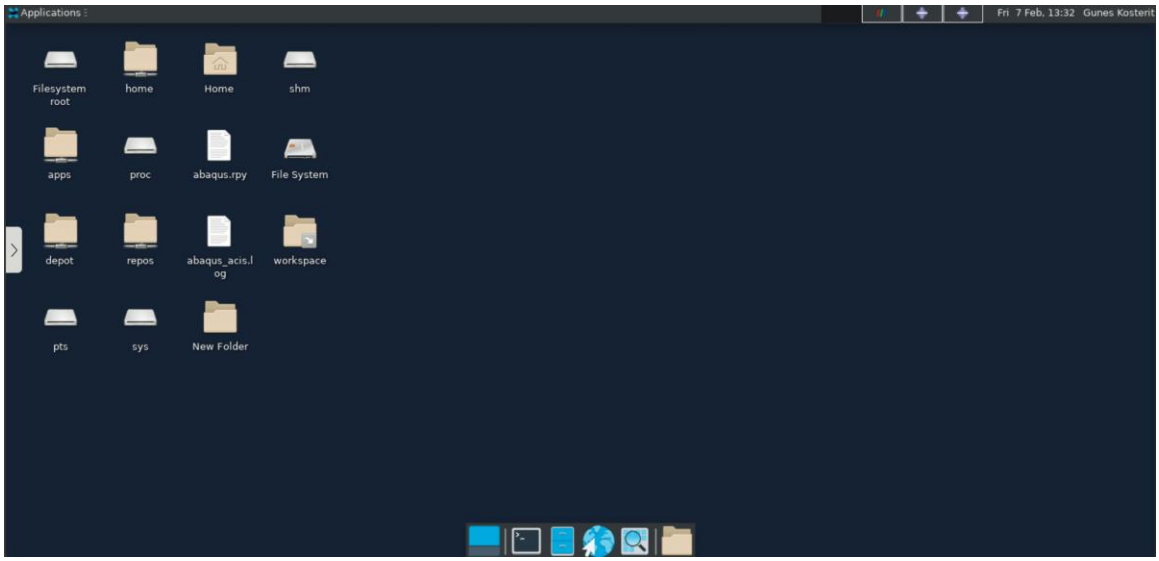
The remote desktop method offers an intuitive, browser-based interface that allows users to access the cluster directly from Purdue's web portal. This option is ideal for users who prefer a graphical interface or need quick access to their computational environment without the need for extensive configuration. The web portal simplifies the connection process, enabling users to monitor jobs, transfer files, and interact with cluster resources in a user-friendly manner.

First go to the website rcac.purdue.edu. Click on the remote desktop.

Enter your purdue username and password and you should receive a duo notification give access to the remote desktop.



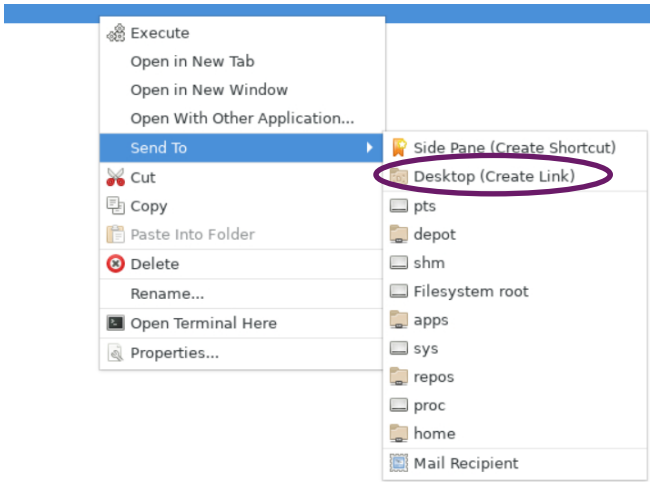
You should see a desktop similar to this. It might not be the same, its ok.



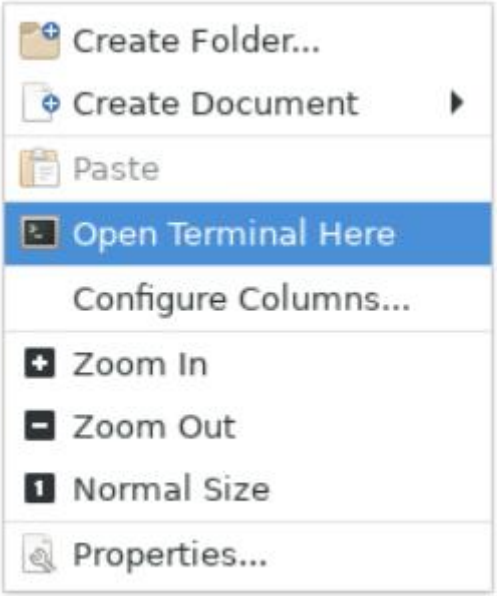
This is a linux based interface. You need to create a work folder. Due to policies and other stuff that I can explain later you should use scratch directory. However, I will demonstrate a simple method to help you use the cluster more efficiently. Open any folder. I clicked home folder.

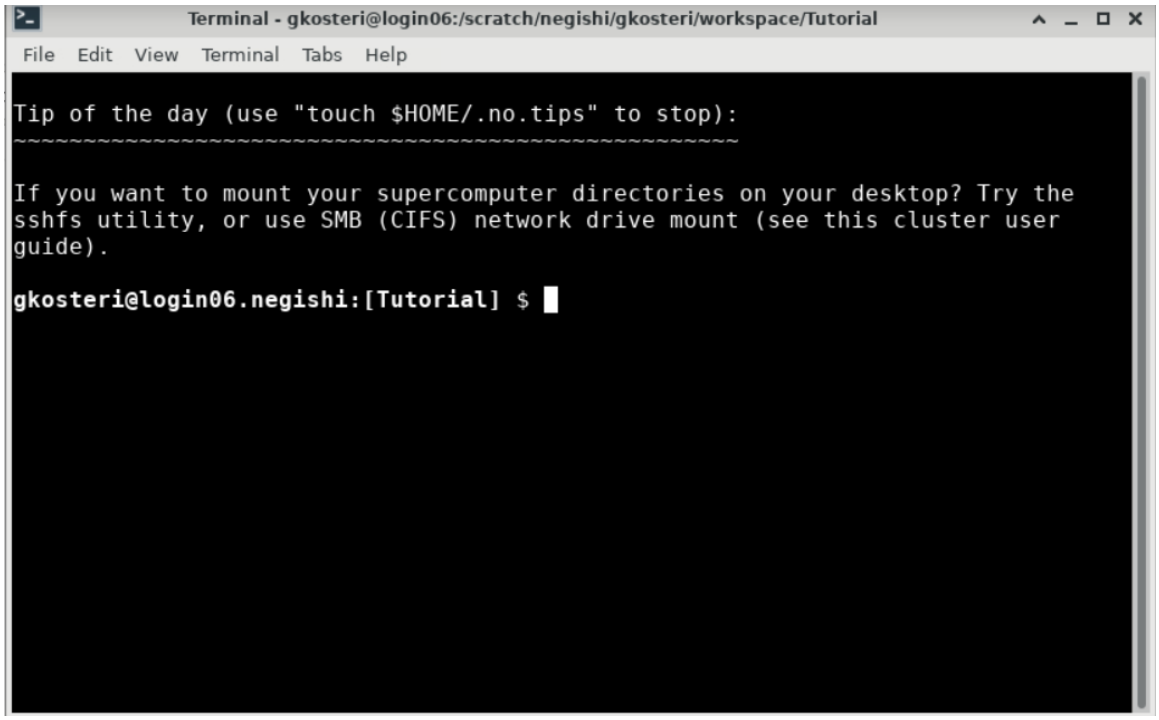
You can see this is the address of the current folder we want to go to scratch. You can either directly paste ”/scratch/negishi/mperesdu/” or navigate to base directory, delete the home/gkosteri/ (whatever you have there) and navigate manually.

Now right click and create a folder called “workspace” here.

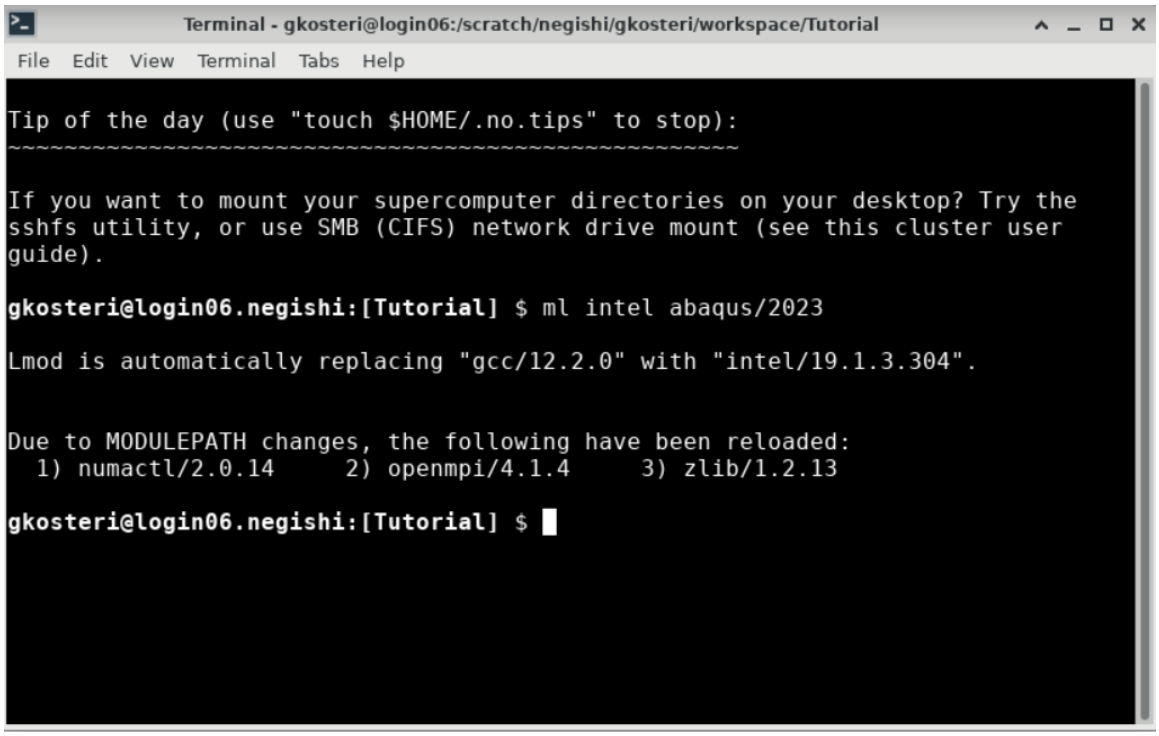


Now you successfully have a folder to work in. In this folder create a empty folder called tutorial. In this folder right click and click on “open terminal here”. This will launch a terminal ON THE CURRENT FOLDER. There are commands and cool stuff you can do with terminal but you can explore those your self with chatgpt or smth.

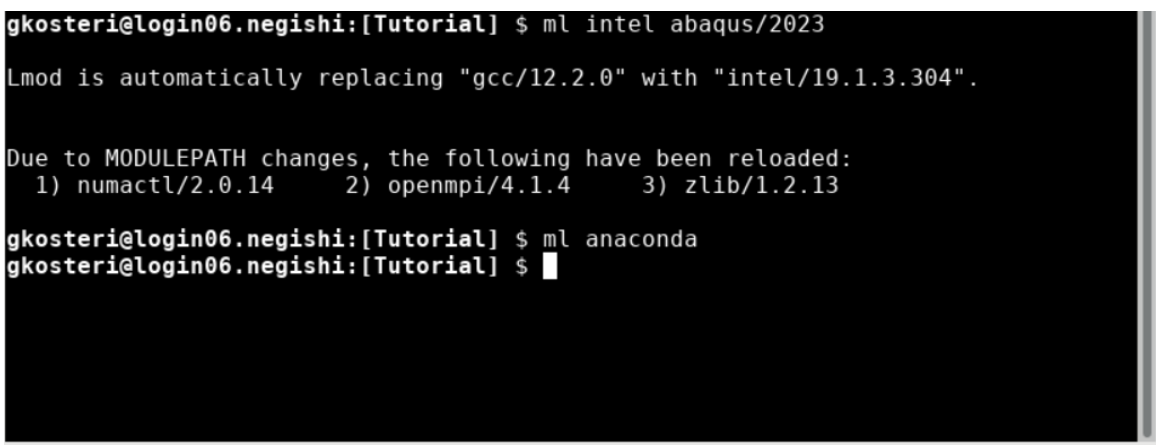




The terminal will always greet you with a tip, its ok. Now we are going to launch abaqus. Type “ml intel abaqus/2023” ml is short for module load. This loads abaqus to the current terminal you will need to load it again if you use a new terminal.



Then further load anaconda module with “ml anaconda”, this helps you use some new python libraries with in the terminal. Your terminal should look like this.



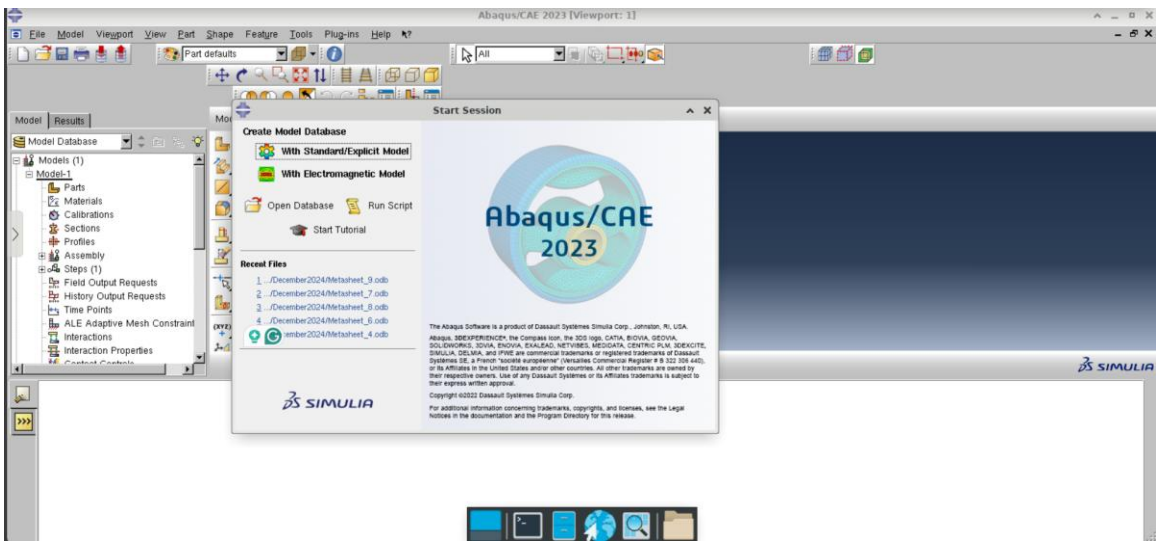
Then launch abaqus. To launch abaqus use “abaqus cae -mesa”. This should write the following and will launch abaqus gui.

```
gkosteri@login06.negishi:[Tutorial] $ ml intel abaqus/2023

Lmod is automatically replacing "gcc/12.2.0" with "intel/19.1.3.304".


Due to MODULEPATH changes, the following have been reloaded:
  1) numactl/2.0.14     2) openmpi/4.1.4     3) zlib/1.2.13

gkosteri@login06.negishi:[Tutorial] $ ml anaconda
gkosteri@login06.negishi:[Tutorial] $ abaqus cae -mesa
Abaqus License Manager checked out the following license:
"cae" from Flexnet server mooring.ecn.purdue.edu
<46 out of 60 licenses remain available>.
```



If you see this screen. You launched abaqus successfully you can use it as if you have it installed in your computer. You can run anything you like you cannot harm our runs in anyway. Just make sure you enter a sensible number of CPUs while running your codes. Even if you wrote a obnoxious number of CPUs it is fine we will let you know and fix it.

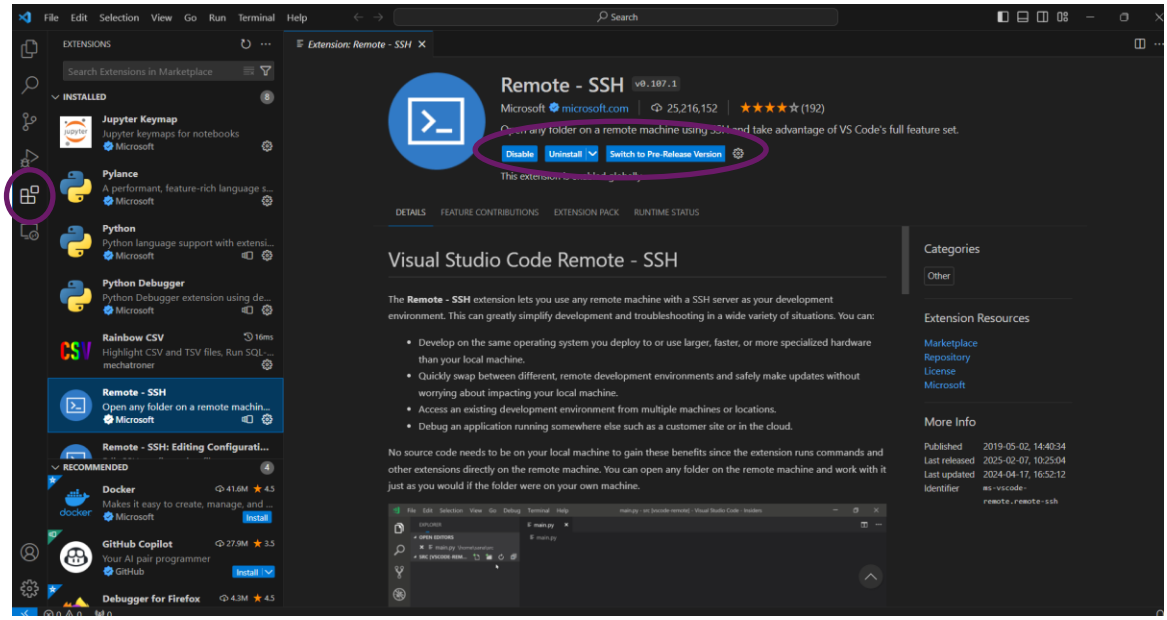
GUI is great for visualization and result processing but for abaqus scripting which is a crucial tool for us we as a group prefer to use something called SSH to connect to cluster and code.

SSH Using VS Code

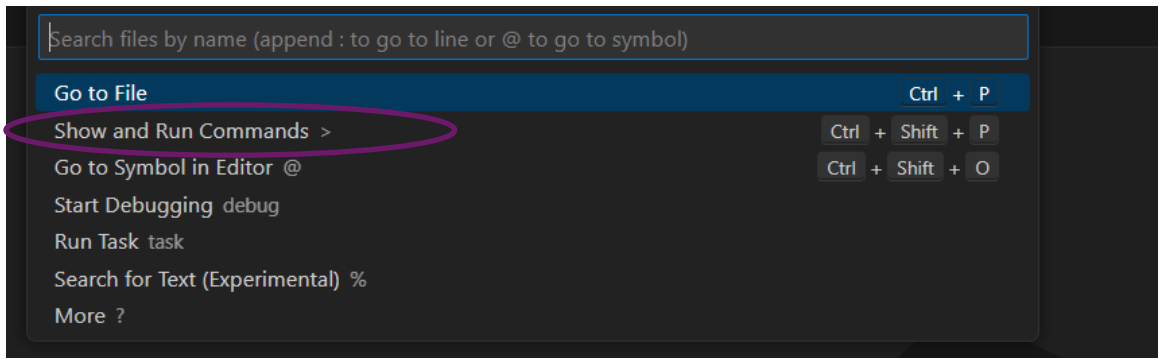
For more advanced users or those requiring command-line access, **Secure Shell (SSH)** via Visual Studio Code is a powerful and flexible option. With VS Code's Remote Development extensions, users can establish an SSH connection to the clusters, providing direct access to terminal commands, file structures, and scripts. This method is well-suited for developers and researchers comfortable with coding environments, enabling seamless integration with version control systems and other development tools.

In the subsequent sections, we will provide detailed instructions on setting up and using both access methods, helping you to choose the one that best fits your workflow. You can install VS Code to YOUR computer through [Download Visual Studio Code - Mac, Linux, Windows](). I assume you will be able to install.

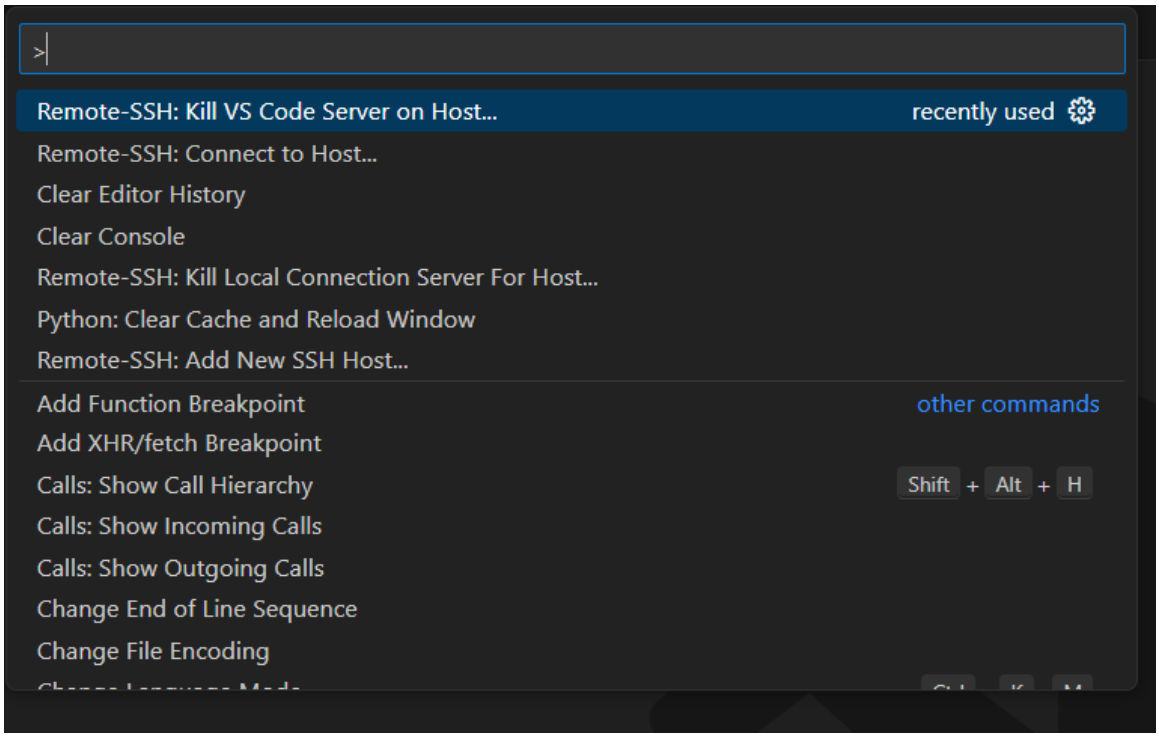
Go and install SSH



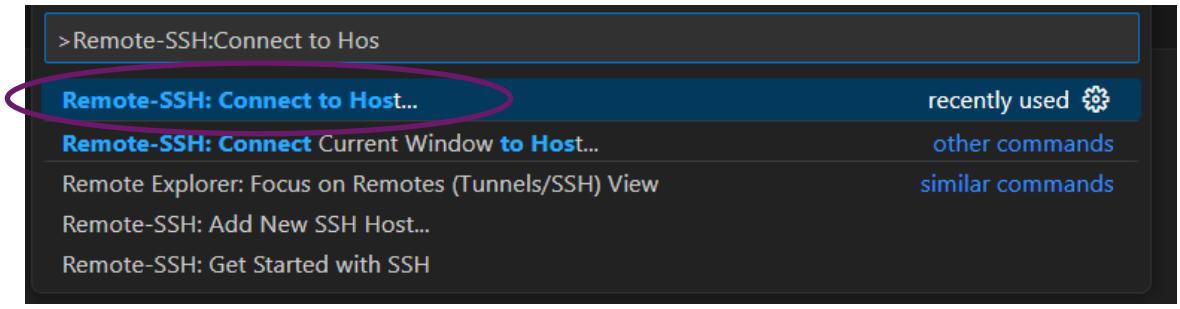
Then press the search bar at the top and click on run command. I am not going through shortcuts because I know you have a mac and I don't know mac.



After clicking on run command you should see ">" symbol pop up.

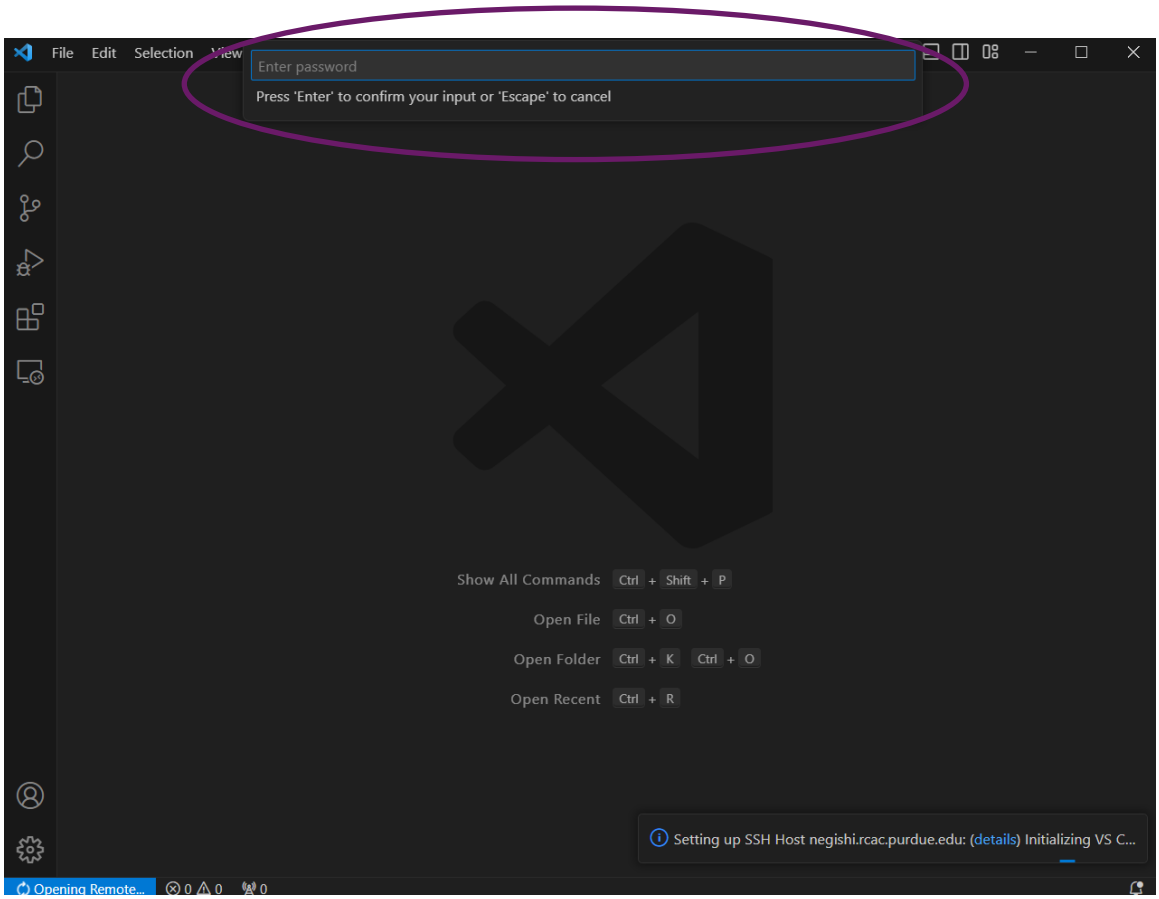


Type the following and you should see:

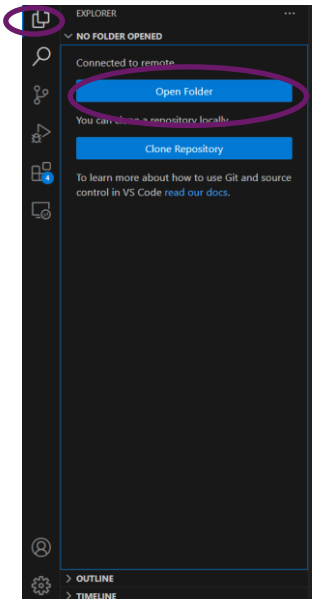


Enter mperesdu@negishi.rcac.purdue.edu as host name.

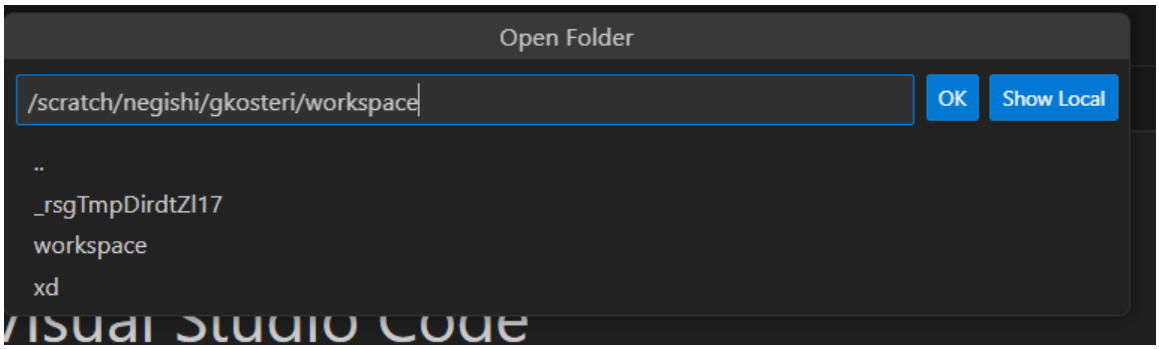
It should ask for your password enter and verify from duo.



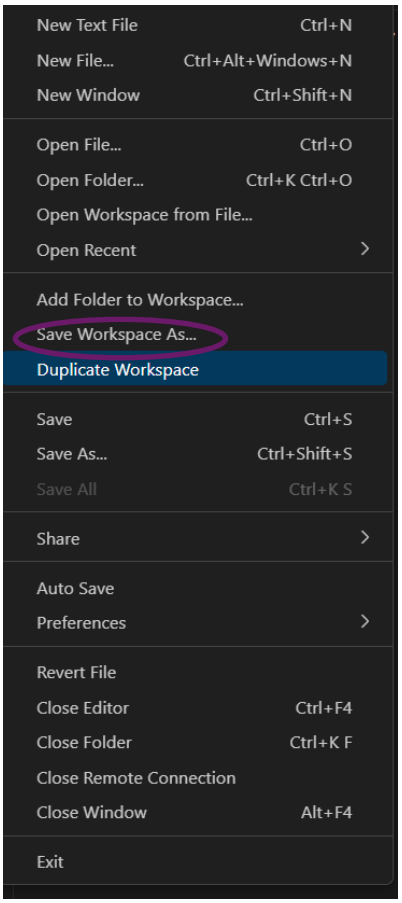
Now you should see a window connecting to cluster. We will need to open a work folder. Click on explorer and add folder as shown below.



Type /scratch/negishi/mperesdu/workspace



Congrats! You should be in your workspace folder now. This is the folder we created in remote desktop. Crazy I know. Now you should be able to see the folder we created as tutorial. If you move any file to this folder in vs code it will be on remote desktop. CS people are crazy in 2025. Move the .py file I sent to this folder and you can edit it using VS code. You can also make cool stuff with the interpreter. If you don't know what this is just ask chatgpt and how to use it. Now save this workspace to your desktop so you don't have to setup every time.



Abaqus Scripting

Abaqus scripting enables users to automate and customize simulations in the Abaqus finite element analysis (FEA) software suite. By leveraging Python, the scripting language supported by Abaqus, users can create, modify, and run simulations programmatically. This approach not only increases efficiency but also provides flexibility for handling complex workflows, batch processing, and repetitive tasks.

Abaqus scripting can be used to define geometry, assign material properties, apply boundary conditions, and control simulation steps, among other tasks. The scripting interface is particularly powerful when working with large models or when multiple scenarios need to be analyzed systematically.

In this section, we will introduce the basics of Abaqus scripting, including:

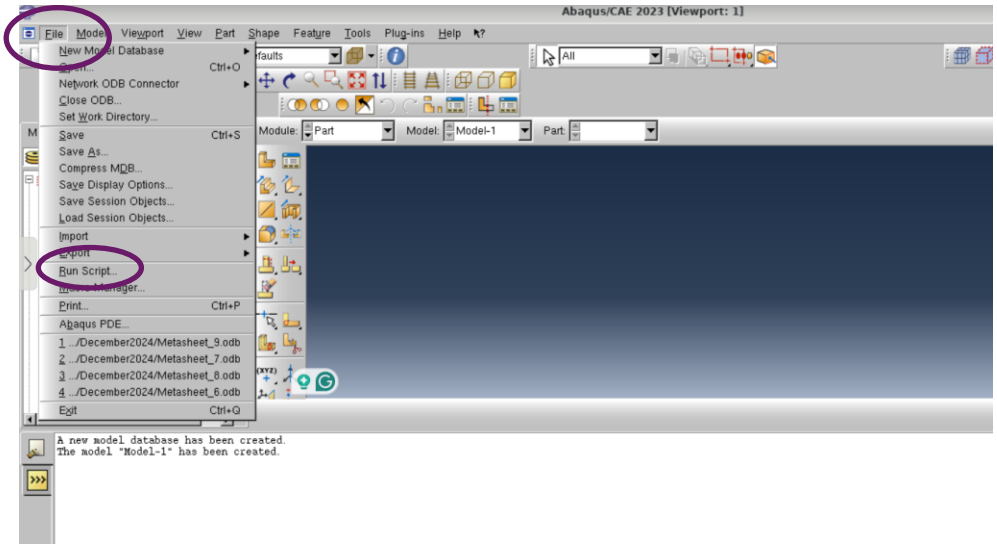
1. **Setting up the Python scripting environment**: Understanding how to access and use Abaqus' built-in Python interpreter.
2. **Key commands and structures**: An overview of the most commonly used functions and modules.
3. **Practical examples**: Step-by-step demonstrations of scripting for typical Abaqus tasks.

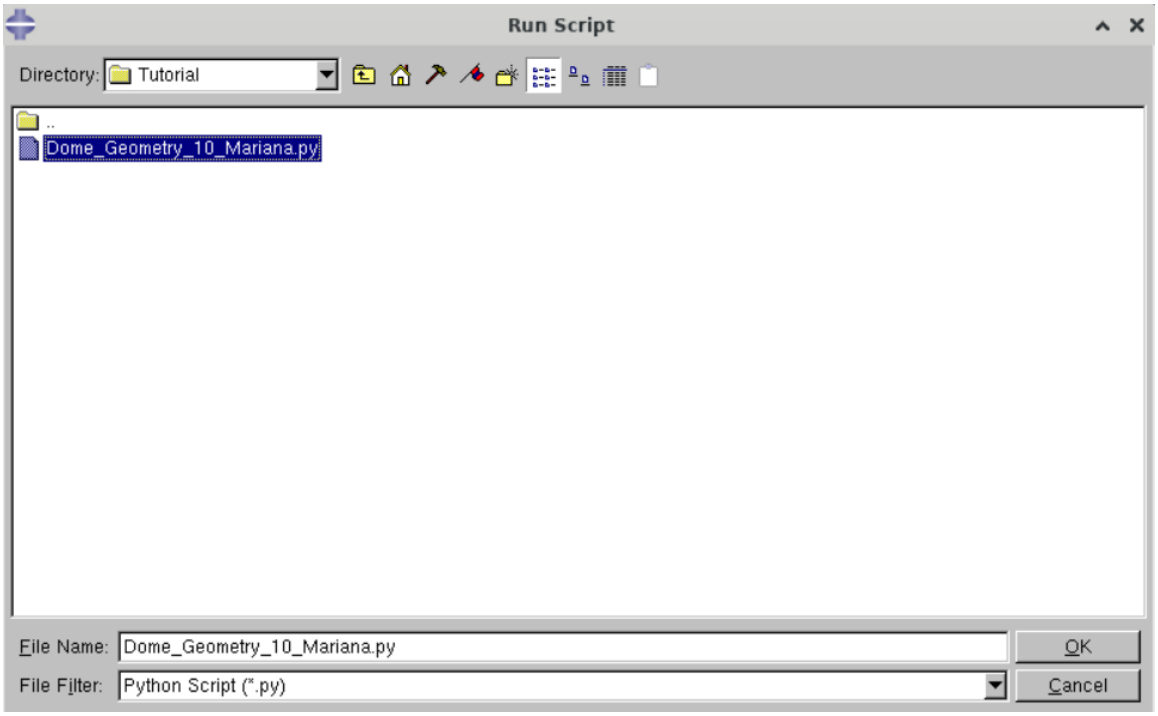
Whether you're new to scripting or looking to enhance your current workflows, this section will provide a comprehensive foundation to get you started with Abaqus scripting.

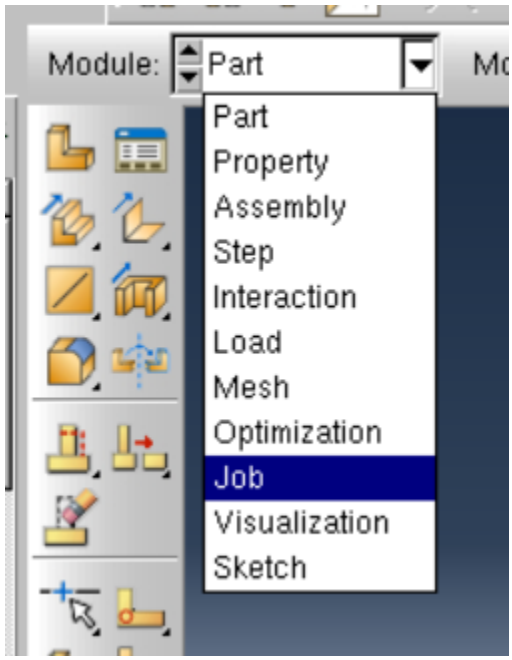
GUI

Running Jobs from GUI

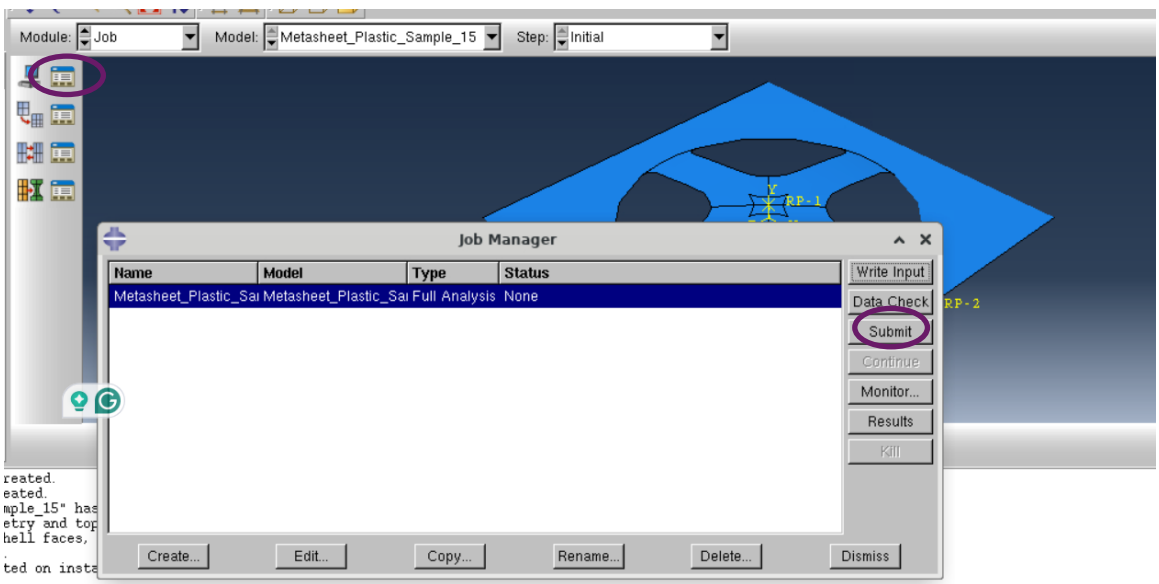
The code I sent you is written in a special way for abaqus to understand you can run it by the following.


Abaqus/CAE 2023 [Viewport: 1]
File
Model
Viewport
View
Part
Shape
Feature
Tools
Plug-ins
Help
New Model Database
Ctrl+O
Network ODB Connector
Close ODB...
Set Work Directory...
Save
Ctrl+S
Save As...
Compress MDB...
Save Display Options...
Save Session Objects...
Load Session Objects...
Import
Export
Run Script...
Print...
Ctrl+P
Abaqus PDE...
1 .../December2024/Metasheet_9.odb
2 .../December2024/Metasheet_7.odb
3 .../December2024/Metasheet_8.odb
4 .../December2024/Metasheet_6.odb
Exit
Ctrl+Q
All
Module: Part
Model: Model-1
Part:
A new model database has been created.
The model "Model-1" has been created.


Run Script
Directory: Tutorial
..
Dome_Geometry_10_Mariana.py
File Name: Dome_Geometry_10_Mariana.py
OK
File Filter: Python Script (*.py)
Cancel



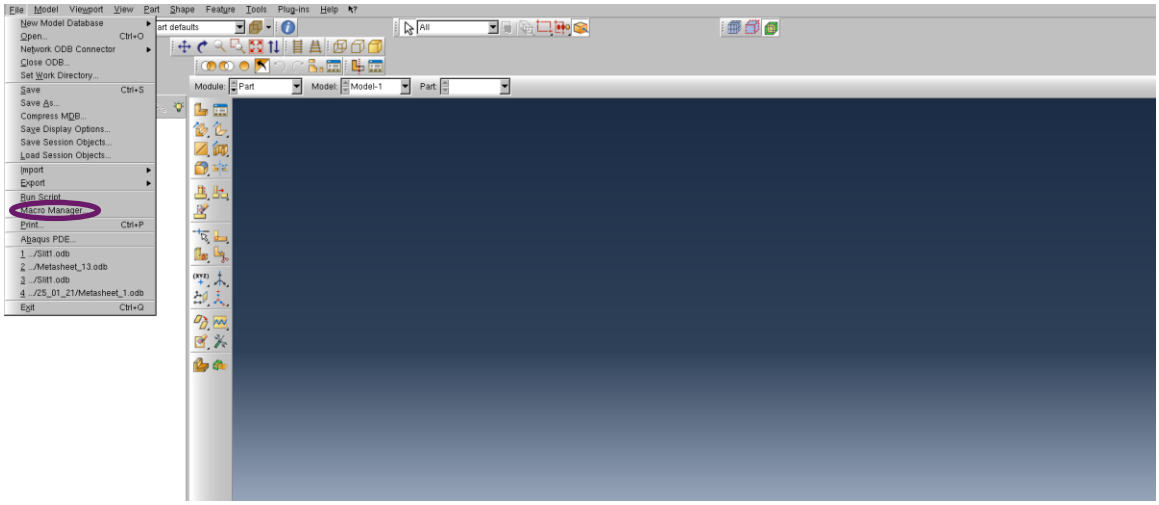
Open the job and submit it to start the run. The status will be “completed” when done.

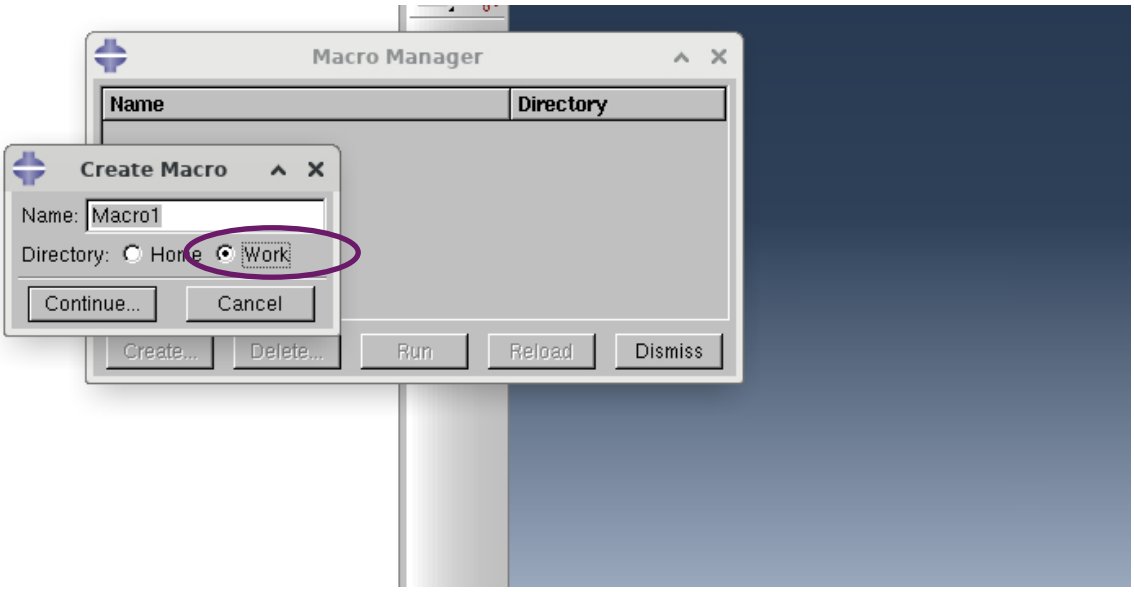


Now go through the code with the help of chat gpt. Don’t send the code to chat gpt just ask what does this function do and ask functions names and try to understand each line except meshing. Meshing is not important.

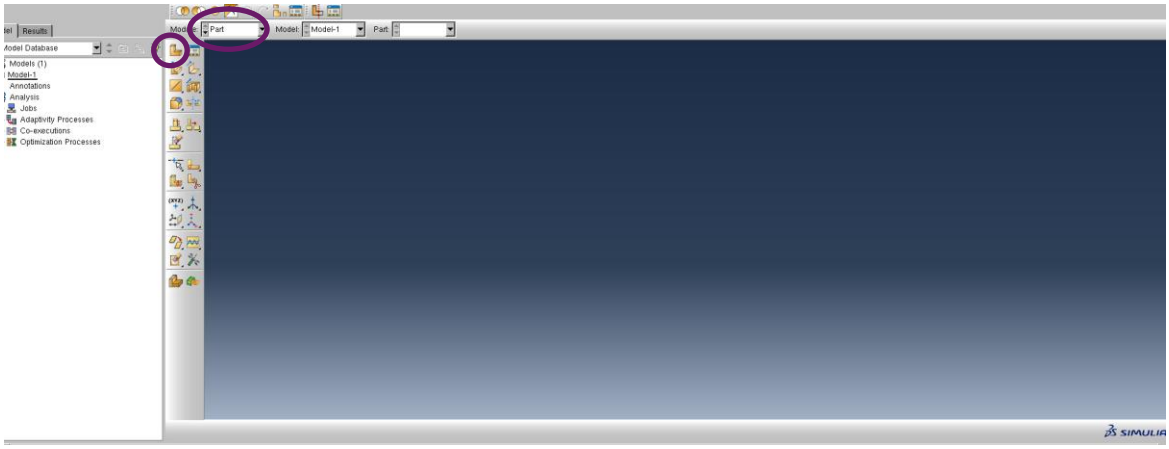
Learning Scripting through macros

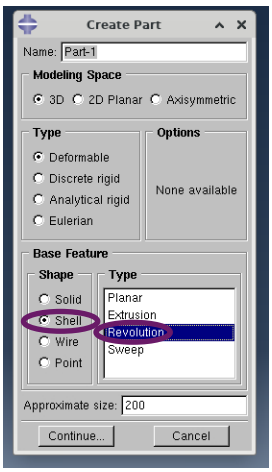
You can learn how to come up with the script for a task that you know how to do in gui using macros. To demonstrate this lets draw a sphere from GUI and then turn this operation to a script. First start recording the macro. Make sure you save it in work directory.



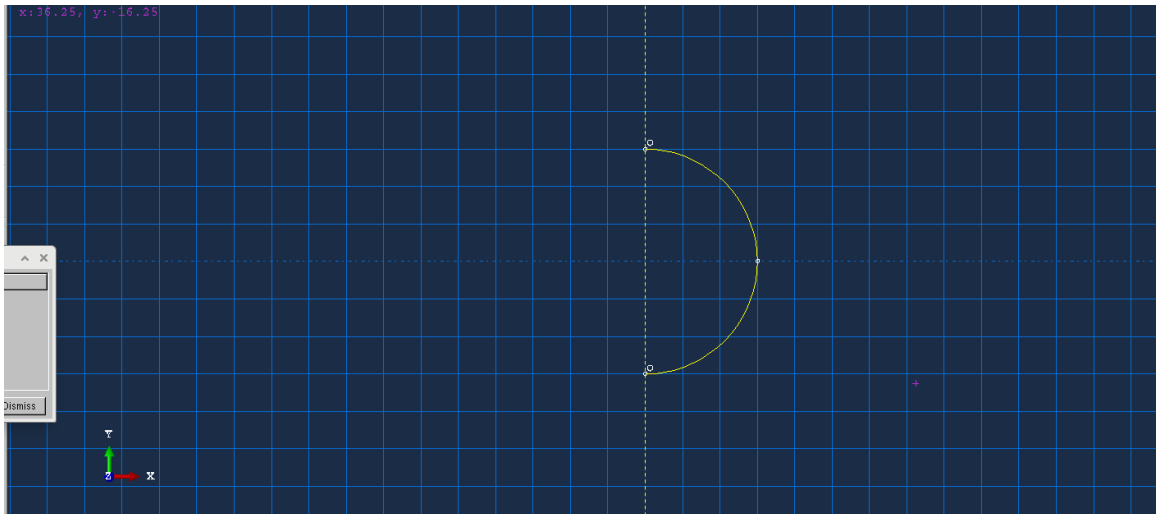


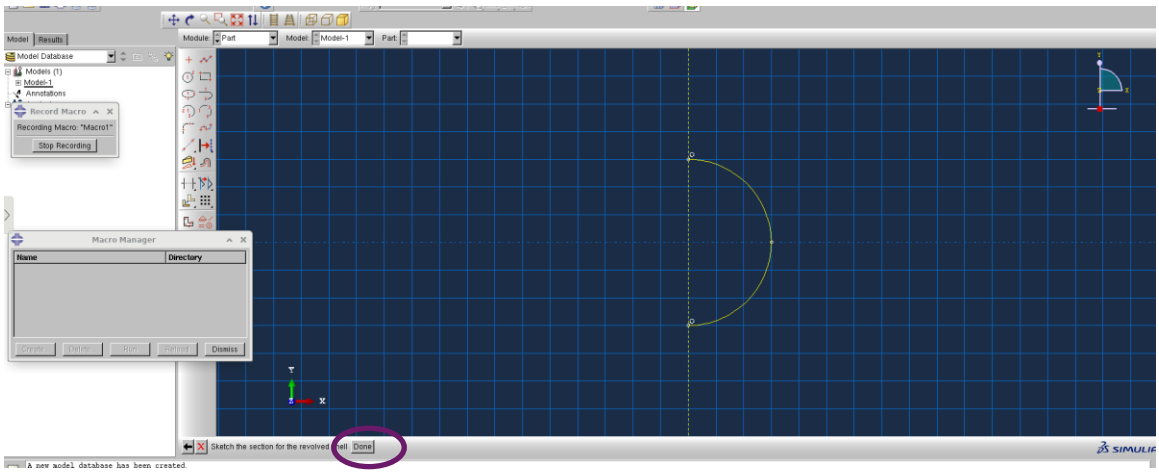
This shows that macro is recording every thing you do on your screen. Yes everything. Even you move your screen it will record that. So, when you try to 'learn' from the script please make sure you understand what every line does. Now lets draw the sphere.



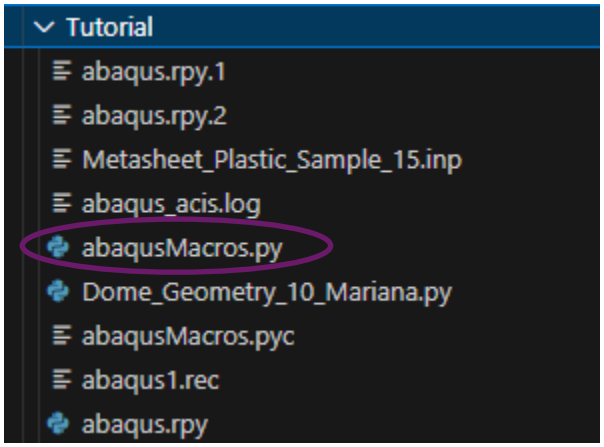


Now you should see a sketch window draw your sphere.





Click "Done" and make angle of rotation 360 degrees. Now you should have a sphere now stop recording the macro. You should see abaqusMacros.py these are the macros you recorded.

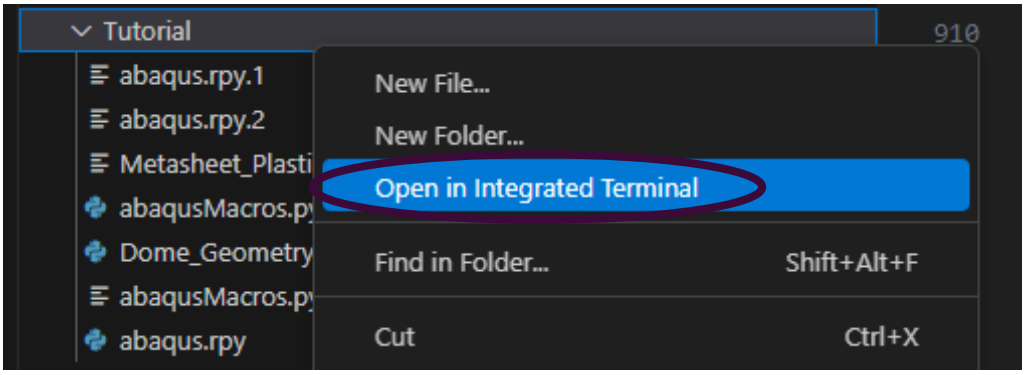


```
def Macro1():
    import section
    import regionToolset
    import displayGroupMdbToolset as dgm
    import part
    import material
    import assembly
    import step
    import interaction
    import load
    import mesh
    import optimization
    import job
    import sketch
    import visualization
    import xyPlot
    import displayGroupOdbToolset as dgo
    import connectorBehavior
    s = mdb.models['Model-1'].ConstrainedSketch(name='__profile__',
        sheetSize=200.0)
    g, v, d, c = s.geometry, s.vertices, s.dimensions, s.constraints
    s.setPrimaryObject(option=STANDALONE)
    s.ConstructionLine(point1=(0.0, -100.0), point2=(0.0, 100.0))
    s.FixedConstraint(entity=g[2])
    s.undo()
    s.Arc3Points(point1=(0.0, 15.0), point2=(0.0, -15.0), point3=(15.0, 0.0))
    s.CoincidentConstraint(entity1=v[0], entity2=g[2], addUndoState=False)
    s.CoincidentConstraint(entity1=v[1], entity2=g[2], addUndoState=False)
    p = mdb.models['Model-1'].Part(name='Part-1', dimensionality=THREE_D,
        type=DEFORMABLE_BODY)
    p = mdb.models['Model-1'].parts['Part-1']
    p.BaseShellRevolve(sketch=s, angle=360.0, flipRevolveDirection=OFF)
    s.unsetPrimaryObject()
    p = mdb.models['Model-1'].parts['Part-1']
    session.viewports['Viewport: 1'].setValues(displayedObject=p)
    del mdb.models['Model-1'].sketches['__profile__']
```

My code looks like this. You can see there are lines such as Arc3points where we draw an arc etc. if you copy these lines to your script and do this in a script!

OPERATING ABAQUS JOBS ON VS CODE DIRECTLY

If you are sure your script is correct you don't need abaqus gui to run your scripts. First open a terminal on VS Code in the work directory you desire.



You should see the directory of the terminal as well as all the terminals open right now. You can have several terminals open at once.

This terminal works as if you opened a terminal on the remote desktop. Any operation you do here runs on the cluster. You can run your scripts here after you load the required modules by typing "abaqus cae NOGUI='Your_script.py' "change Your_script.py part with your script name.

This will create a .inp file which is the input file that abaqus requires to run jobs. This file has all the information of your problem like mesh, steps and etc. When you submit a job in abaqus GUI what you are doing actually is submitting this .inp file.

Now it's a good time to understand how Purdue HPCs (High Performance computer) work. You are given a front end to do your general stuff such as operating abaqus or running these kind of abaqus scripts. When you submit a job in abaqus GUI you submit this job in the front end you will notice that your remote desktop is running slower. There is also a backend to submit jobs and purdue HPC uses a slurm system. This backend basically an environment that we don't have direct access but you can use shell scripts to send jobs to backend. In the backend we have 256 CPUs and 220 abaqus simulation tokens. These resources enables us to run simulations in parallel with a lot of CPUs which further enables us to optimize our structures or do parameter sweeps to understand our design domain.

```
#!/bin/bash
#!/bin/sh
#SBATCH -A aarrieta
#SBATCH --nodes=1 --ntasks=15
#SBATCH --time=02:00:00
#SBATCH -J Tutorial-Mariana
#SBATCH -o Tutorial%j.out
#SBATCH -e Tutorial%j.out
#SBATCH --mail-user=mperesdu@purdue.edu # Destination email address
#SBATCH --mail-type=END,FAIL # Event(s) that triggers email notification
(BEGIN,END,FAIL,ALL)
#SBATCH --array 1-1%12
# IMPORTANT! This gives you the paralell runs --> array initial-last%max
jobs at a time

# Go where we were when we typed 'squeue'
if [[ -n $SLURM_SUBMIT_DIR ]]; then
    cd $SLURM_SUBMIT_DIR
fi

echo "Simulation had started for ${obj_type} Objects"
echo "Starting in: $(pwd)"

# -------------------------------------------------------
# Load Modules
# -------------------------------------------------------
module load intel abaqus/2023
unset SLURM_GTIDS

# -------------------------------------------------------
# Set ABAQUYS JOB
# -------------------------------------------------------
n=$SLURM_ARRAY_TASK_ID # Get n simulation value from array
```

```
job_name="Metasheet_Plastic_Sample_15"
inp_name="Metasheet_Plastic_Sample_15.inp"

abaqus interactive job=${job_name} inp=${inp_name} cpus=$SLURM_NTASKS
scratch=$PWD
echo "JOB IS DONE!"
```

This is an example bash script. To submit one job to backend. You can copy this and save as backend.sh. Then you can run it by typing sbatch backend.sh. It will give you this job's job number.

Once you submit your job. You can track your job in two main ways. Firstly, you can write to any terminal (terminal in any directory) "squeue -A aarrieta" and it will show you the current jobs we are running as a group on the current HPC.

```
gkosteri@login01.negishi:[workspace] $ squeue -A aarrieta
JOBID        USER        ACCOUNT     NAME             NODES  CPUS  TIME_LIMIT ST TIME
22509643     jrivaspa    aarrieta    matlab-launch        1    16 14-00:00:00  R 3-07:26:31
22569279_1   gkosteri    aarrieta    WR-CombinedSlits     1    30    12:00:00  R 2:33:56
```

At the time I was preparing this document you can see that me and Jose were running some jobs and I was using 30 CPUs and he was using 16.

The second way is a better way to see how soon you job is going to end. In the current work directory the job you submitted will create a .sta (status file)

```
Abaqus/Standard 2023                    DATE 16-Feb-2025 TIME 21:15:36
SUMMARY OF JOB INFORMATION:
STEP  INC ATT SEVERE EQUIL TOTAL  TOTAL      STEP       INC OF       DOF    IF
              DISCON ITERS ITERS  TIME/    TIME/LPF    TIME/LPF    MONITOR RIKS
              ITERS               FREQ
  1     1   1     1     2     3  0.0100     0.0100     0.01000
  1     2   1     1     0     1  0.0200     0.0200     0.01000
  1     3   1     2     3     5  0.0350     0.0350     0.01500
  1     4   1     1     1     2  0.0575     0.0575     0.02250
  1     5   1     1     4     5  0.0912     0.0912     0.03375
  1     6   1U    4     6    10  0.0912     0.0912     0.05063
  1     6   2     1     5     6  0.104      0.104      0.01266
```

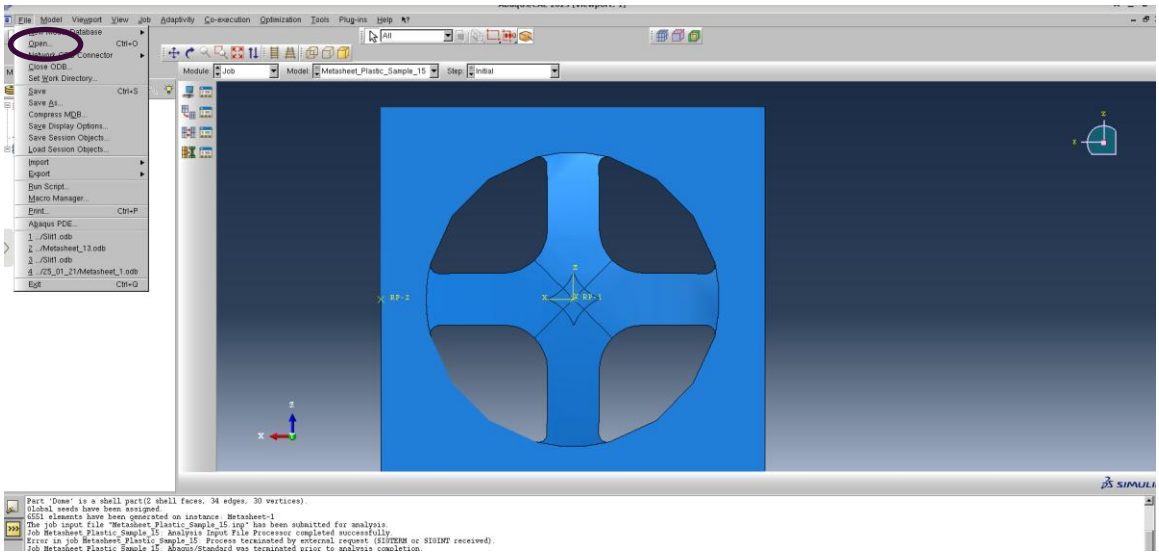
This file will show you the increment and steps of your simulation. If you have a good understanding of the job you are running, you can observe which step and time your simulation currently running. This file is extremely useful since you can see which time steps are hard to solve so you can reflect on that on your other simulations.

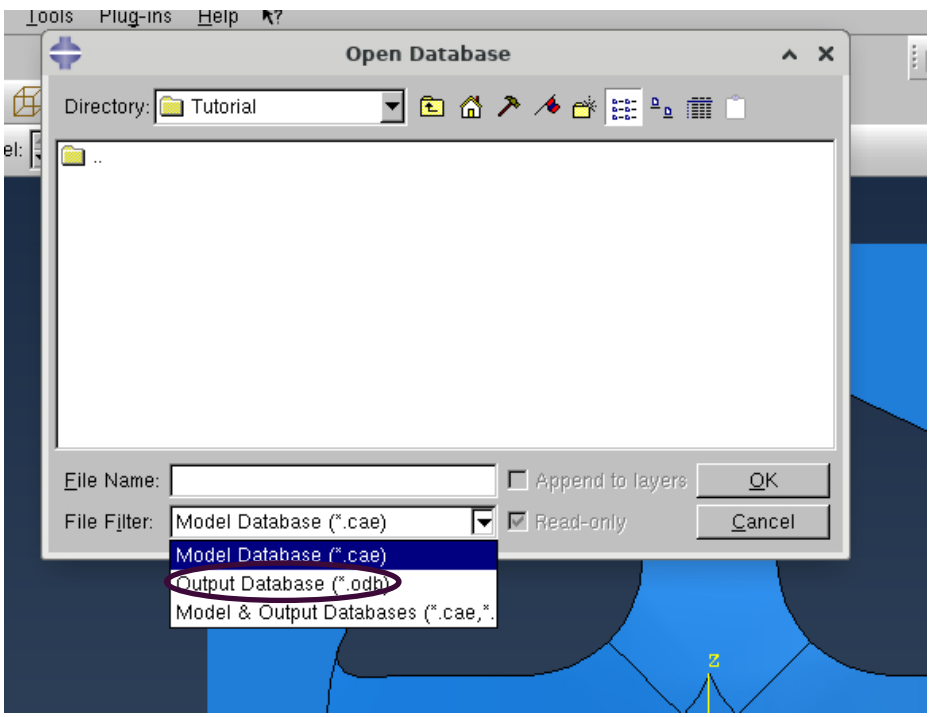
RESULTS POST PROCESSING

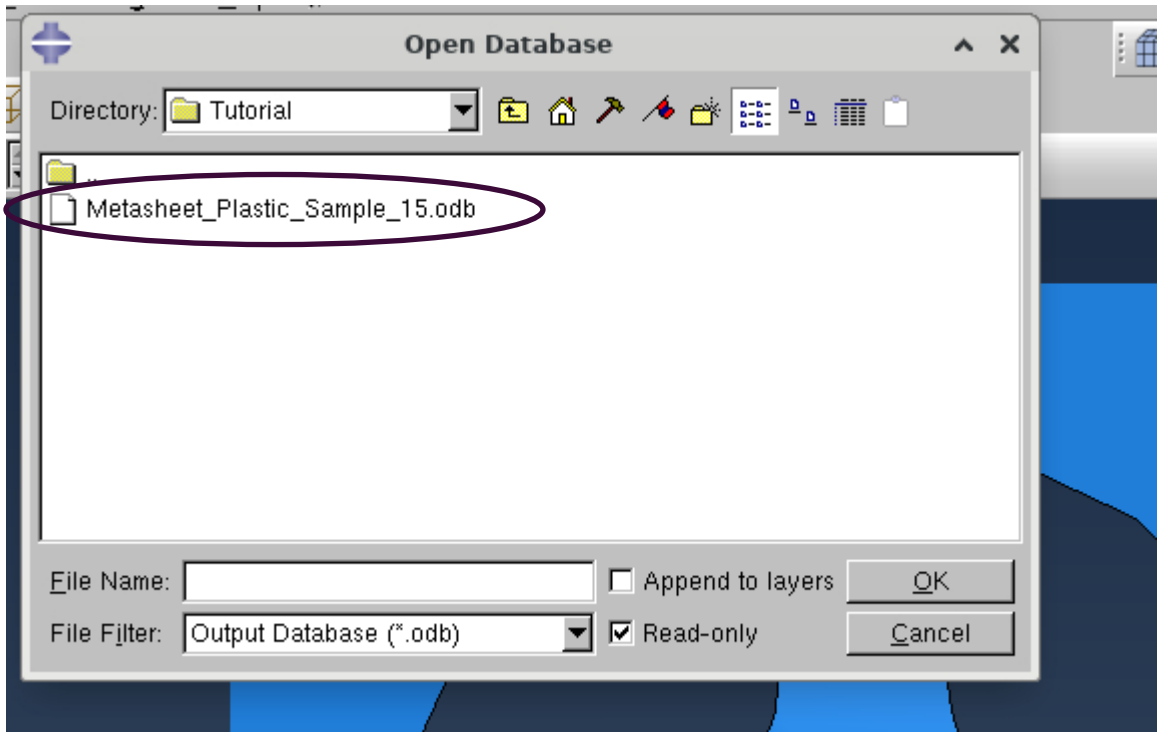
Post Processing for a Single Simulation

All your simulation data is saved inside the .odb file. This file will be updated every few steps of simulation. You can select what kind of outputs you require in your .py script. I will go

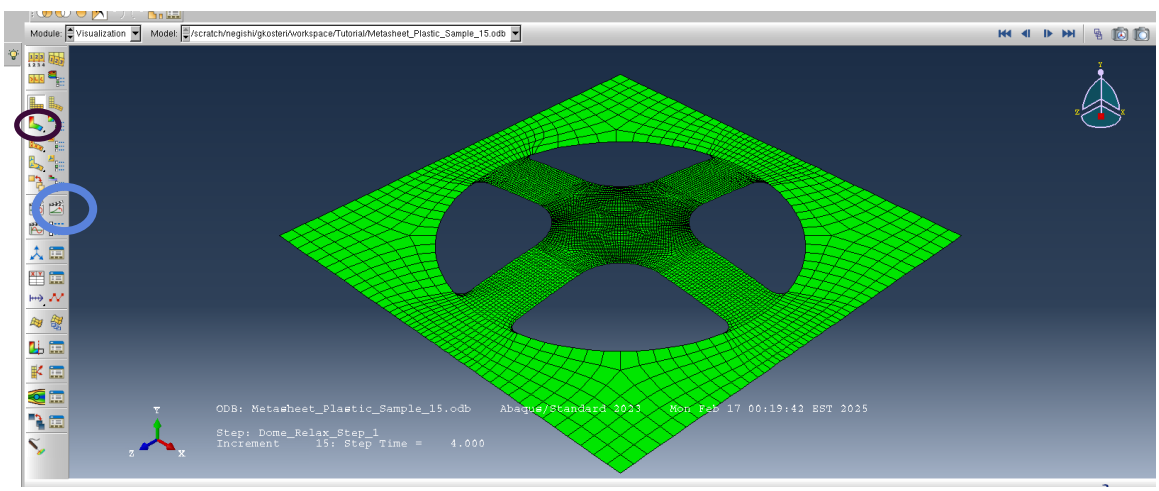
through the code I sent you last time. Once your job is done open your .odb file. This odb file can be from a simulation that was run on backend or frontend does not matter.





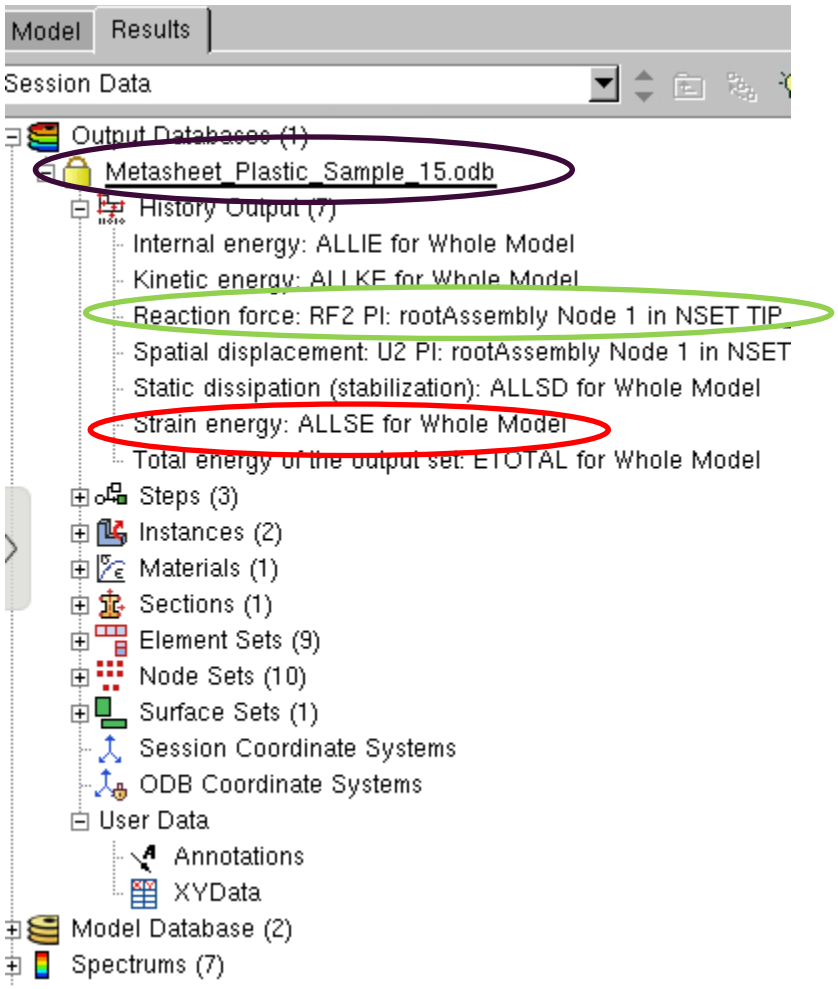


At this point there are so many things I can show that it is better for you to spent some time in this interface to become more familiar with it.



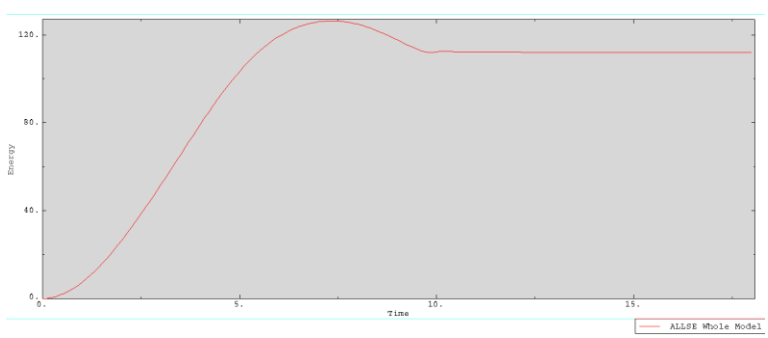
The colors are by default von mises strain you can change from upper right corner.

To simulate the code you can click on the button circled with blue. Now that you can see the animation, you can also see the specific outputs we asked from the simulation.



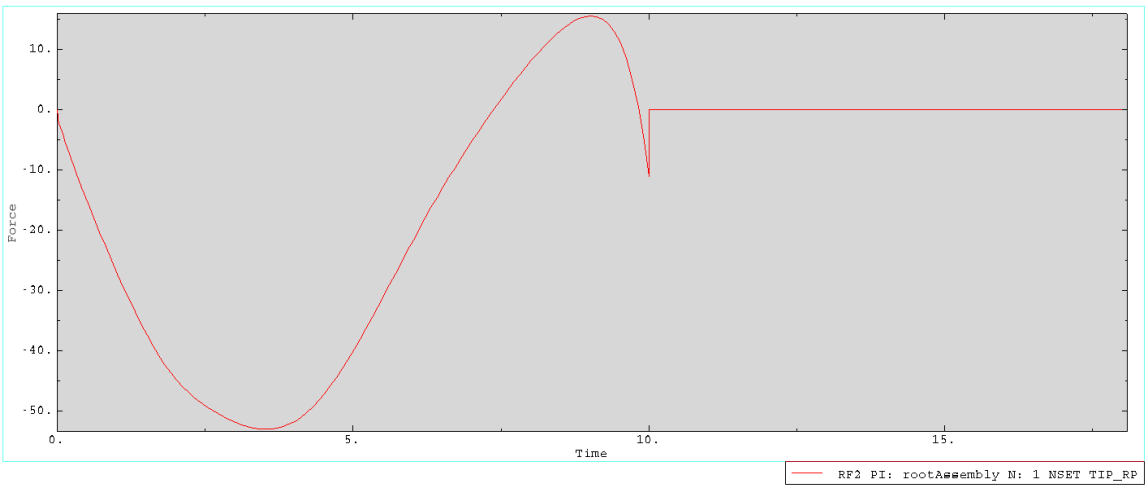
The red result:

One of the most important things in our simulations is strain energy. This scalar value of our total system shows the energy we gave to the system and trapped energy throughout the simulation. We can discuss more on this on our meetings.



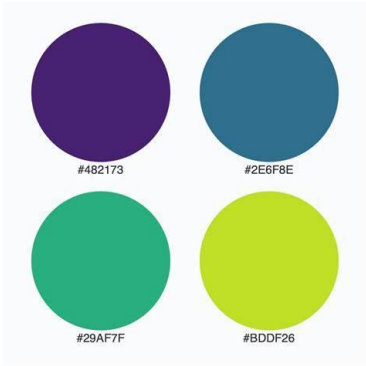
The green result:

RF2 means reaction force in the second coordinate which is Y for our system. This data shows us the force required to apply the velocity boundary condition to invert the dome. This data is very critical because later we will use this and strain energy to measure bistability of our dome.

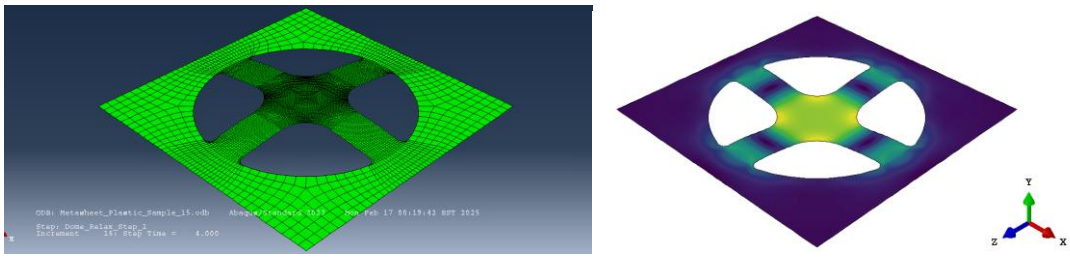


Making your abaqus results look better

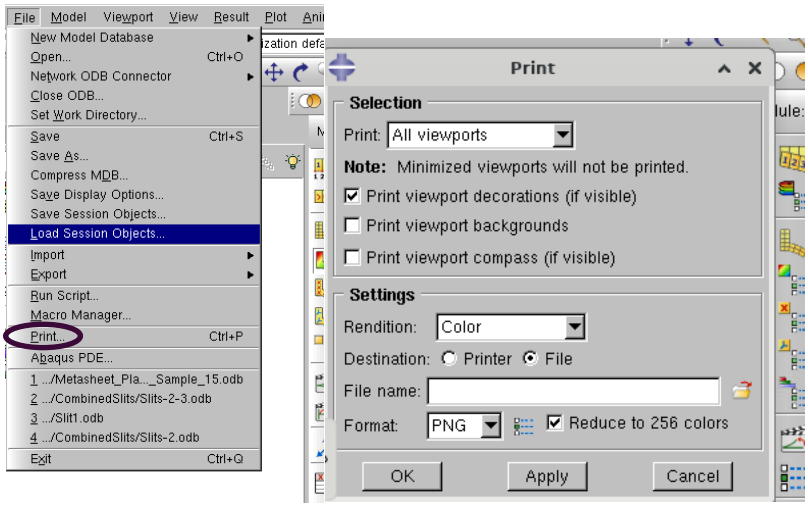
Juan had a code that he shared with me that changes you abaqus visualization module similar to the plots we use called Post_Image_ABAQUS 3.py. For example we use viridis color palate on our plots. This code will make your abaqus use the same color palate so that you can extract images and animations better. See viridis color palatte below.



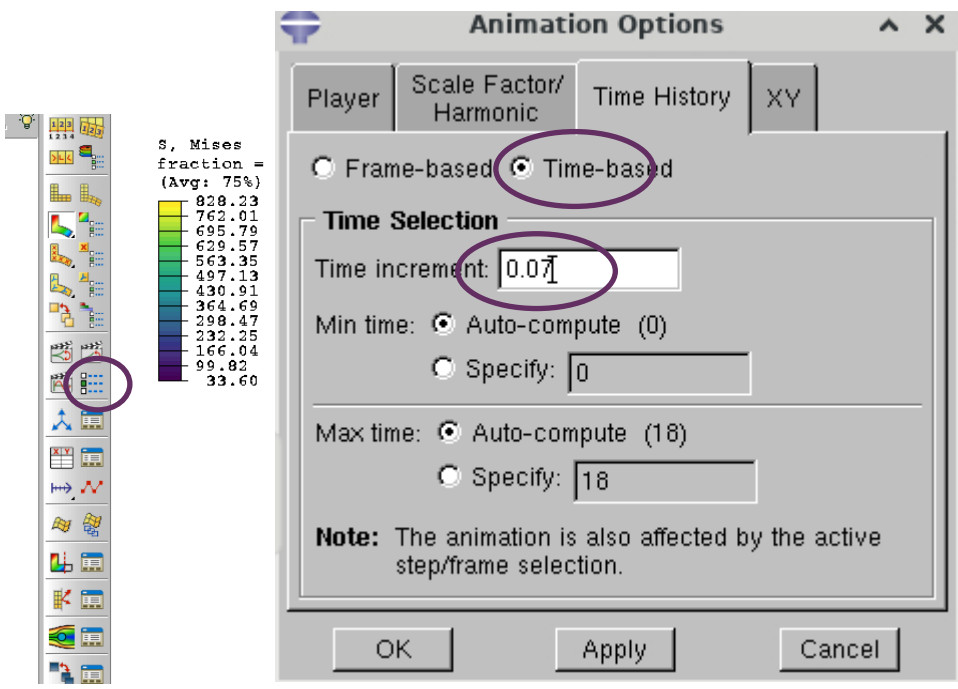
If you run this code when a odb file is open it should look like this.



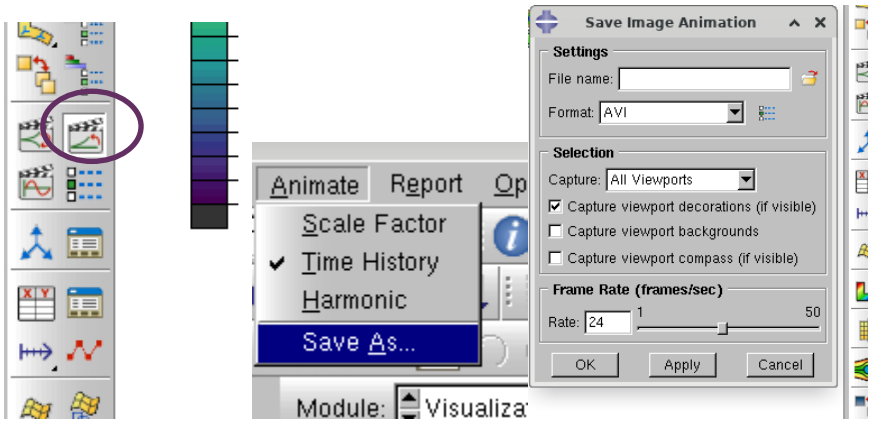
When you are extracting an image you should use print function



When you want to save an animation. You should use



Then start the animation and go to the animate pane. Then follow the commands.



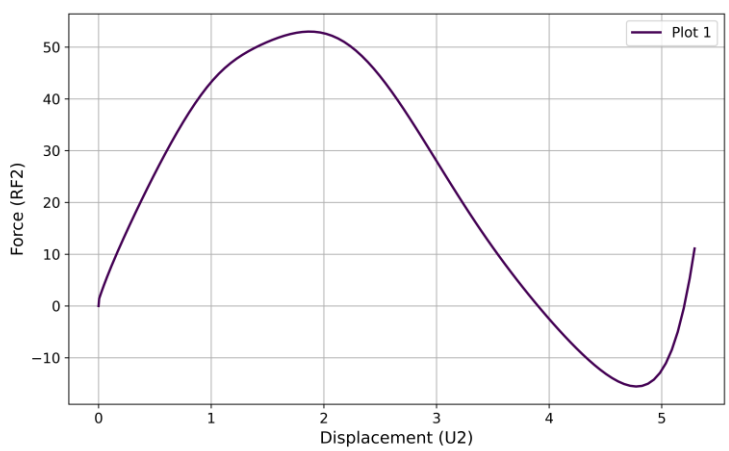
Mass Post Processing for Simulations in Batches

The script 'Maxstressstrainforarun.py' is an abaqus script that will extract the selected outputs from the odb file. It should be run as

abaqus cae NOGUI='Maxstressstrainforarun.py'

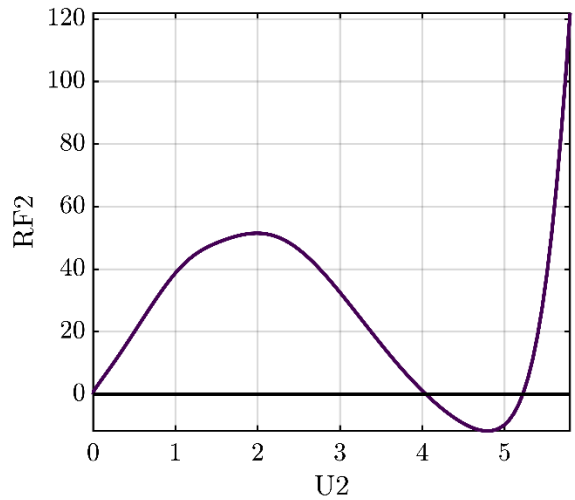
```
Time,MaxStress,MaxStrain,ALLSE,U2,RF2
0.0,0,0,0.0,0.0,-0.0
0.00999999977648,4.03160476685,1.85993430932e-05,0.000962727644946,-0.00529199978337,-1.40729987621
0.019999999553,8.05540657043,3.70850902982e-05,0.00377475423738,-0.0105839995667,-1.75494921207
0.035000000149,14.0145654678,6.47095585009e-05,0.0114320972934,-0.0185219999403,-2.25698852539
0.0575000010431,22.7795333862,0.000105867795355,0.0304940026253,-0.0304290000349,-2.99123573303
0.0912500023842,35.5630226135,0.000166780708241,0.0756086036563,-0.0482895001769,-4.05556201935
```

It should generate a csv file that looks like this. Then you can use the python script named 'plotter_stress_strain.py' to plot and get this plot.



This plot is not exactly what you should use in your presentations but to see the results directly from the VS code. If you have several odb files in the same directory these will plot all of them on top of each other and create a csv file for all the odbs.

For plots that you are going to use on your presentations you should use SingleInversion.m matlab file to create better plots that look like this. You can add legend as accordingly. If you need to plot several on top of each other I can help with that or you can try to figure out yourself.



When you are using these plots copy from the png file that matlab creates. It has more DPI and higher quality.